1. Swim Times.
This is a program written using Donald Knuth's literate programming paradigm (see more below). It fetches best short-course yard (SCY) times for four particular swimmers:

Stella Julianna Evans — 10 Under Girls
Kalea Rose Benavente — 13–14 Girls
Kenneth Ray Evans — 11–12 Boys
Keith Santiago Evans — 11–12 Boys

Twelve events are reported for each swimmer: 50, 100, 200, and 500 Freestyle; 50 and 100 Butterfly; 50 and 100 Backstroke; 50 and 100 Breaststroke; and 100 and 200 Individual Medley. Each time record includes the swim date and motivational standard attained (B, BB, A, . . .). Data is fetched live from the USA Swimming data hub, which is powered by the Sisense analytics platform.

2. How it works. The USA Swimming data hub exposes a Sisense JAQL API. We make two kinds of requests:

1. A *person search* to resolve the swimmer's internal *PersonKey*, given a search string and a substring to match the full name.
2. A *times query* for each event, filtered by *PersonKey* and event code (e.g. `100 FR SCY`). Each row delivers five cells: swim time, sort key, meet name, swim date (as `YYYYMMDD`), and motivational standard.

All HTTP communication is handled by `libcurl`. JSON responses are scanned with simple string operations rather than a full parse tree.

3. Literate programming. Donald Knuth introduced *literate programming* in 1984 as a way of writing software that is meant to be read by human beings first and executed by computers second. Rather than annotating code with comments, a literate program interweaves prose and code in a single source document. The prose explains the *why*—the motivation, the design decisions, the mathematical reasoning—while the code expresses the *how*. The two live together in one file (conventionally given the extension `.w`) and are separated only at build time by two companion tools: `ctangle` and `cweave`.

4. ctangle. `ctangle` is the *tangling* tool. It reads a `.w` source file and extracts the C code sections, assembling them in the order dictated by named chunk references rather than the order in which they appear in the document. The result is a plain `.c` file that a standard C compiler can process without any knowledge of literate programming. In this project, running

```
ctangle stella-times.w
```

produces `stella-times.c`, which is then compiled with `cc` and linked against `libcurl` to create the `stella-times` executable. The generated `.c` file should be treated as a build artefact: the `.w` file is the true source of record.

5. cweave. `cweave` is the *weaving* tool. It reads the same `.w` source and produces a `.tex` file formatted for `pdftex` using the `cwebmac` macro package. `cweave` pretty-prints all C code with bold keywords, italic identifiers, and cross-references, and numbers every named chunk so the reader can follow the program's logical structure independently of its physical layout. An index of identifiers and a table of contents are generated automatically. Running

```
cweave stella-times.w
```

produces `stella-times.tex`; running `pdftex` on that file yields the typeset documentation you are reading now.

6. Compilation. After tangling with `ctangle`:

```
gcc -O2 -o swim-times swim-times.c $(curl-config --libs)
```

7. Program structure. The top-level arrangement of the tangled output is:

⟨ Includes 9 ⟩
⟨ Global constants 10 ⟩
⟨ Option flags 11 ⟩
⟨ Type definitions 13 ⟩
⟨ HTTP utilities 16 ⟩
⟨ JSON scanner 20 ⟩
⟨ Person lookup 23 ⟩
⟨ Times fetch 25 ⟩
⟨ Main function 29 ⟩

8. Includes and constants.

9. ⟨ Includes 9 ⟩ ≡

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <curl/curl.h>
```

This code is used in section 7.

10. The Sisense bearer token authenticates us against the analytics instance that backs `data.usaswimming.org`. `EVENTS` lists the twelve SCY event codes we query.

⟨ Global constants 10 ⟩ ≡

```
#define NUM_EVENTS 12
#define MAX_TIMES 200
  static const char SISENSE_TOKEN[] = "eyJhbGciOiJIUzI1NiI\
      sInR5cCI6IkpXVCJ9"".eyJ1c2VyIjoiNjY0YmE2NmE5M2ZiYTUwMDM4NWIyMWQwIiwiYXBpU2Vjcm\
      V0Ijo""iNDQ0YTE3NWQtM2I1OC03NDhhLTVlMGEtYTVhZDE2MmRmODJlIiwiYWxsb3\
      dlZFRl""bmFudHMiOlsiNjRhYzE5ZTEwZTkxNzgwMDFiYzM5YmVhIl0sInRlbmFudEl\
      kIjoiN""jRhYzE5ZTEwZTkxNzgwMDFiYzM5YmVhIn0"".izSIvaD2udKTs3QRngla1Aw23kZVyoq7Xh23Ab\
      PUw1M";
  static const char SISENSE_API[] = "https://usaswimming.sisense.com/api/datasources";
  static const char *EVENTS[NUM_EVENTS] = {"50 FR SCY", "100 FR SCY", "200 FR SCY",
      "500 FR SCY", "50 FL SCY", "100 FL SCY", "50 BK SCY", "100 BK SCY", "50 BR SCY",
      "100 BR SCY", "100 IM SCY", "200 IM SCY"};
```

This code is used in section 7.

11. The program accepts two optional flags on the command line.

The `-o` flag takes a comma-separated list of one or more keywords. Six keywords are recognised:

- `stella` — output only Stella Julianna Evans' events.
- `kalea` — output only Kalea Rose Benavente's events.
- `kenny` — output only Kenneth Ray Evans' events.
- `keith` — output only Keith Santiago Evans' events.
- `fastest` — print only the single fastest time per event.
- `csv` — emit CSV lines (swimmer name on every line) instead of a table.

Keywords may be combined, e.g. `-o stella,fastest` or `-o kalea,csv`. When no swimmer keyword is specified all four swimmers are shown.

The `-e` flag takes one or more comma-separated SCY event codes (e.g. `-e "100 FR SCY,50 FL SCY"`) and restricts output to those events only. The flag may also be repeated (e.g. `-e "100 FR SCY" -e "50 FL SCY"`). When `-e` is omitted all twelve events are reported.

If no options are given at all the program prints a usage message and exits.

⟨ Option flags 11 ⟩ ≡

```
#define OPT_STELLA (1 << 0)    /* restrict output to Stella's events */
#define OPT_KALEA (1 << 1)    /* restrict output to Kalea's events */
#define OPT_FASTEST (1 << 2)    /* print only the single fastest time */
#define OPT_CSV (1 << 3)    /* emit CSV lines instead of a table */
#define OPT_KENNY (1 << 4)    /* restrict output to Kenneth Ray Evans */
#define OPT_KEITH (1 << 5)    /* restrict output to Keith Santiago Evans */
  static int g_opts = 0;    /* output-selection flags; 0 = show everything */
  static char g_events[NUM_EVENTS][32];    /* event codes requested via -e */
  static int g_nevents = 0;    /* number of entries in g_events */
```

This code is used in section 7.

12. Data structures.

13. A *Buffer* holds a dynamically-grown heap string. `libcurl` appends each response chunk to it via the write callback.

⟨ Type definitions 13 ⟩ ≡

```
typedef struct {
  char *data;
  size_t size;
} Buffer;
```

See also section 14.

This code is used in section 7.

14. A *TimeRow* records one swim result. *sort_key* is a numeric proxy for the swim time (smaller is faster); *time* is the formatted string (e.g. `1:02.45`); *date* is the swim date formatted as `YYYY-MM-DD`; *standard* is the motivational standard attained (e.g. `B`, `BB`, `A`, `Slower Than B`); and *meet* is the meet name. The Sisense API returns the sort key as a quoted decimal string such as `"1010007029.00"`, so we store it as a **double**.

⟨ Type definitions 13 ⟩ +≡

```
typedef struct {
  double sort_key;
  char time[32];
  char date[16];
  char standard[48];
  char meet[256];
} TimeRow;
```

15. HTTP utilities.

16. The `libcurl` write callback appends each incoming chunk to a **Buffer**, growing the allocation as needed. It maintains a null terminator so the buffer can always be treated as a C string.

⟨ HTTP utilities 16 ⟩ ≡

```
static size_t write_cb(void *ptr, size_t size, size_t nmemb, void *ud)
{
  Buffer *b = (Buffer *) ud;
  size_t n = size * nmemb;
  char *p = realloc(b->data, b->size + n + 1);
  if (¬p) return 0;
  b->data = p;
  memcpy(b->data + b->size, ptr, n);
  b->size += n;
  b->data[b->size] = '\0';
  return n;
}
```

See also section 17.

This code is used in section 7.

17. *post_json* performs a single HTTP POST with a JSON body and returns the response as a null-terminated heap string, or Λ on failure. The caller is responsible for freeing the returned string.

⟨ HTTP utilities 16 ⟩ +≡

```
static char *post_json(const char *url, const char *body)
{
  CURL * curl = curl_easy_init();
  if (¬curl) return Λ;
  Buffer buf = {Λ, 0};
  char auth_hdr[1600];
  snprintf(auth_hdr, sizeof auth_hdr, "Authorization:␣Bearer␣%s", SISENSE_TOKEN);
  struct curl_slist *hdrs = Λ;
  hdrs = curl_slist_append(hdrs, "Content-Type:␣application/json");
  hdrs = curl_slist_append(hdrs, auth_hdr);
  curl_easy_setopt(curl, CURLOPT_URL, url);
  curl_easy_setopt(curl, CURLOPT_HTTPHEADER, hdrs);
  curl_easy_setopt(curl, CURLOPT_POSTFIELDS, body);
  curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, write_cb);
  curl_easy_setopt(curl, CURLOPT_WRITEDATA, &buf);
  CURLcode rc = curl_easy_perform(curl);
  curl_slist_free_all(hdrs);
  curl_easy_cleanup(curl);
  if (rc ≠ CURLE_OK) {
    free(buf.data);
    return Λ;
  }
  return buf.data;
}
```

18. JSON scanner.

19. Sisense JAQL responses contain a top-level `"values"` array. Each element is itself an array of objects, each holding either a `"text"` key (string value) or a `"data"` key (numeric value). A typical person-search row looks like:

```
["text":"Stella Julianna Evans","data":12345,"text":"CAST"]
```

Rather than build a full parse tree we scan forward for a key and extract the immediately following value, advancing a position pointer.

20. *scan_string* finds the first `"key"` at or after $*pos$, copies the string value that follows into *out* (at most $max_len - 1$ bytes), and advances $*pos$ past it. Returns 1 on success, 0 if the key is absent.

⟨ JSON scanner 20 ⟩ ≡

```
static int scan_string(const char *key, const char **pos, char *out, size_t max_len)
{
  char needle[128];
  snprintf(needle, sizeof needle, "\"%s\"", key);
  const char *p = strstr(*pos, needle);
  if (¬p) return 0;
  p += strlen(needle);
  while (*p ≡ '␣' ∨ *p ≡ ':') p++;
  if (*p ≠ '"') return 0;
  p++;
  const char *e = strchr(p, '"');
  if (¬e) return 0;
  size_t len = (size_t)(e − p);
  if (len ≥ max_len) len = max_len − 1;
  memcpy(out, p, len);
  out[len] = '\0';
  *pos = e + 1;
  return 1;
}
```

See also section 21.

This code is used in section 7.

21. *scan_long* is the numeric counterpart: it finds `"key"`, converts the digits that follow to a **long**, stores the result in $*out$, and advances $*pos$.

⟨ JSON scanner 20 ⟩ +≡

```
static int scan_long(const char *key, const char **pos, long *out)
{
  char needle[128];
  snprintf(needle, sizeof needle, "\"%s\"", key);
  const char *p = strstr(*pos, needle);
  if (¬p) return 0;
  p += strlen(needle);
  while (*p ≡ '␣' ∨ *p ≡ ':') p++;
  char *end;
  *out = strtol(p, &end, 10);
  if (end ≡ p) return 0;
  *pos = end;
  return 1;
}
```

22. Person lookup. *lookup_person_key* posts a search for *search_query* and scans the result rows for one whose full name contains *match_substr* (after lower-casing). It returns a heap-allocated decimal *PersonKey* string and, optionally, the swimmer's full name in $*$*out_name*. Returns Λ on failure.

23.

⟨ Person lookup 23 ⟩ ≡

```
static char *lookup_person_key(const char *search_query, const char *match_substr, const char
        **out_name)
{
  char url[512];
  snprintf(url, sizeof url, "%s/aPublicIAAaPersonIAAaSearch/jaql", SISENSE_API);
  char body[1024];
  snprintf(body, sizeof body,
      "{""\"datasource\":{""\"title\":\"Public␣Person␣Search\",""\"fullname\":\"Loca\
      lHost/Public␣Person␣Search\"},""\"metadata\":[""{\"jaql\":{\"table\\
      ":\"Persons\",\"column\":\"FullName\",""\"dim\":\"[Persons.\
      FullName]\",\"datatype\":\"text\",""\"title\":\"Name\",""\"filter\":{\"conta\
      ins\":\"%s\"}}},""{\"jaql\":{\"table\":\"Persons\",\"colu\
      mn\":\"PersonKey\",""\"dim\":\"[Persons.PersonKey]\",\"datat\
      ype\":\"numeric\",""\"title\":\"PersonKey\"}},""{\"jaql\":{\"table\\
      ":\"Persons\",\"column\":\"ClubName\",""\"dim\":\"[Persons.ClubName]\",\"dataty\
      pe\":\"text\",""\"title\":\"Club\"}}""],\"count\":100,\"offset\":0}", search_query);
  char *resp = post_json(url, body);
  if (¬resp) {
    fputs("Error:␣person␣lookup␣request␣failed\n", stderr);
    return Λ;
  }
  char *key = Λ;
  char *name = Λ;
  const char *p = strstr(resp, "\"values\"");
  while (p) {
    char full_name[256];
    if (¬scan_string("text", &p, full_name, sizeof full_name)) break;
        /* Lower-case the name and check for the match substring. */
    char lower[256];
    size_t flen = strlen(full_name);
    for (size_t i = 0; i ≤ flen; i++)
      lower[i] = (char)(full_name[i] ≥ 'A' ∧ full_name[i] ≤ 'Z' ? full_name[i] + 32 : full_name[i]);
    if (strstr(lower, match_substr)) {
      long pk;
      if (scan_long("data", &p, &pk)) {
        char tmp[32];
        snprintf(tmp, sizeof tmp, "%ld", pk);
        key = strdup(tmp);
        name = strdup(full_name);
      }
      break;
    }
  }
  free(resp);
  if (¬key) fprintf(stderr, "Error:␣swimmer␣\"%s\"␣not␣found\n", search_query);
  if (out_name) *out_name = name;
  else free(name);
```

```
    return key;
  }
```

This code is used in section 7.

24. Times fetch.

25. *insertion_sort* sorts n **TimeRow** records in-place by *sort_key* ascending (smallest = fastest time). A swimmer's career rarely exceeds a few dozen entries per event, so $O(n^2)$ is entirely adequate.

⟨ Times fetch 25 ⟩ ≡

```
static void insertion_sort(TimeRow *rows, int n)
{
  for (int i = 1; i < n; i++) {
    TimeRow tmp = rows[i];
    int j = i - 1;
    while (j ≥ 0 ∧ rows[j].sort_key > tmp.sort_key) {
      rows[j + 1] = rows[j];
      j--;
    }
    rows[j + 1] = tmp;
  }
}
```

See also sections 26 and 27.

This code is used in section 7.

26. *format_date* converts an 8-digit `YYYYMMDD` string (such as `20230305`) to `YYYY-MM-DD` format in *out* (which must be at least 11 bytes).

⟨ Times fetch 25 ⟩ +≡

```
static void format_date(const char *ymd8, char *out)
{
  if (strlen(ymd8) ≡ 8) {
    out[0] = ymd8[0];
    out[1] = ymd8[1];
    out[2] = ymd8[2];
    out[3] = ymd8[3];
    out[4] = '-';
    out[5] = ymd8[4];
    out[6] = ymd8[5];
    out[7] = '-';
    out[8] = ymd8[6];
    out[9] = ymd8[7];
    out[10] = '\0';
  }
  else {
    strncpy(out, ymd8, 10);
    out[10] = '\0';
  }
}
```

27. *fetch_times* retrieves all SCY times for *person_key* in *event_code*, sorts them fastest-first, and prints them. Each JAQL result row delivers five cells in order: time, sort key, meet name, swim date (as `YYYYMMDD`), and motivational standard type. The sort key (a decimal float string such as `"1010007029.00"`) is converted to **double** with *strtod*. The date is reformatted from `YYYYMMDD` to `YYYY-MM-DD` by *format_date*. *swimmer_name* is the resolved full name of the swimmer (used in CSV output). When *opts* has `OPT_CSV` set the output is a comma-separated line per time record with the swimmer name on every line; when `OPT_FASTEST` is set only the single fastest time is printed.

```
⟨ Times fetch 25 ⟩ +≡
  static void fetch_times(const char *person_key, const char *event_code, const char
          *swimmer_name, int opts)
  {
    char url[512];
    snprintf(url, sizeof url, "%s/aUSAIAAaSwimmingIAAaTimesIAAaElasticube/jaql", SISENSE_API);
    char body[2048];
    snprintf(body, sizeof body, "{""\"datasource\":{""\"title\":\"USA␣Swi\
        mming␣Times␣Elasticube\",""\"fullname\":\"LocalHost/USA␣Swimming␣T\
        imes␣Elasticube\"},""\"metadata\":["{\"jaql\":{\"table\":\"UsasSwimTime\",\\
        "column\":\"PersonKey\",""\"dim\":\"[UsasSwimTime.PersonKey]\",\"\
        datatype\":\"numeric\",""\"title\":\"PersonKey\",""\"filter\":{\"equal\
        s\":%s}},\"panel\":\"scope\"},""{\"jaql\":{\"table\":\"SwimEvent\",\"co\
        lumn\":\"EventCode\",""\"dim\":\"[SwimEvent.EventCode]\",\"dat\
        atype\":\"text\",""\"title\":\"Event\",""\"filter\":{\"equals\":\"%s\"}},\"panel\
        \":\"scope\"},""{\"jaql\":{\"table\":\"UsasSwimTime\",""\"column\":\"SwimTi\
        meFormatted\",""\"dim\":\"[UsasSwimTime.SwimTimeFormatted]\",""\"datatype\":\"text\
        \",\"title\":\"Time\"}},""{\"jaql\":{\"table\":\"UsasSwimTime\",\\
        "column\":\"SortKey\",""\"dim\":\"[UsasSwimTime.SortKey]\",\"da\
        tatype\":\"numeric\",""\"title\":\"SortKey\"}},""{\"jaql\":{\"table\\
        ":\"Meet\",\"column\":\"MeetName\",""\"dim\":\"[Meet.Mee\
        tName]\",\"datatype\":\"text\",""\"title\":\"Meet\"}},""{\"jaql\":{\"table\\
        ":\"UsasSwimTime\",""\"column\":\"SeasonCalendarKey\",""\"dim\":\"[UsasSwim\
        Time.SeasonCalendarKey]\",""\"datatype\":\"numeric\",\"title\":\"Da\
        te\"}},""{\"jaql\":{\"table\":\"TimeStandard\",""\"column\":\"Standa\
        rdType\",""\"dim\":\"[TimeStandard.StandardType]\",""\"datatype\":\"text\
        \",\"title\":\"Standard\"}}""],\"count\":100,\"offset\":0}", person_key, event_code);
    char *resp = post_json(url, body);
    if (¬resp) {
      fprintf(stderr, "Error:␣times␣request␣failed␣for␣%s\n", event_code);
      return;
    }
    TimeRow rows[MAX_TIMES];
    int nrows = 0;
    const char *p = strstr(resp, "\"values\"");
      /* Each row has five cells scanned in order via "text": 0 = swim time string, 1 = sort key as decimal
         float string, 2 = meet name, 3 = date as YYYYMMDD, 4 = motivational standard. */
    while (p ∧ nrows < MAX_TIMES) {
      char time_str[32];
      if (¬scan_string("text", &p, time_str, sizeof time_str)) break;
      if (time_str[0] ≡ '\0') break;
      char sort_str[64];
      if (¬scan_string("text", &p, sort_str, sizeof sort_str)) break;
```

```
      char meet_str[256];
      if (¬scan_string("text", &p, meet_str, sizeof meet_str)) break;
      char date_str[16];
      if (¬scan_string("text", &p, date_str, sizeof date_str)) break;
      char std_str[48];
      if (¬scan_string("text", &p, std_str, sizeof std_str)) break;
      rows[nrows].sort_key = strtod(sort_str, Λ);
      strncpy(rows[nrows].time, time_str, sizeof rows[nrows].time − 1);
      rows[nrows].time[sizeof rows[nrows].time − 1] = '\0';
      format_date(date_str, rows[nrows].date);
      strncpy(rows[nrows].standard, std_str, sizeof rows[nrows].standard − 1);
      rows[nrows].standard[sizeof rows[nrows].standard − 1] = '\0';
      strncpy(rows[nrows].meet, meet_str, sizeof rows[nrows].meet − 1);
      rows[nrows].meet[sizeof rows[nrows].meet − 1] = '\0';
      nrows++;
    }
    free(resp);
    insertion_sort(rows, nrows);
    int lim = (opts & OPT_FASTEST) ? (nrows > 0 ? 1 : 0) : nrows;
    if (opts & OPT_CSV) {    /* CSV: swimmer,event,time,date,standard,meet (no header here; caller
          emits the header once before the first swimmer loop). */
      for (int i = 0; i < lim; i++)
        printf("\"%s\",\"%s\",\"%s\",%s,\"%s\",\"%s\"\n", swimmer_name ? swimmer_name : "",
            event_code, rows[i].time, rows[i].date, rows[i].standard, rows[i].meet);
    }
    else {
      printf("%s␣---␣%s:\n", event_code,
          (opts & OPT_FASTEST) ? "fastest␣time" : "all␣times␣(fastest␣first)");
      printf("%-12s␣␣%-10s␣␣%-13s␣␣%s\n", "Time", "Date", "Standard", "Meet");
      printf("%-12s␣␣%-10s␣␣%-13s␣␣%s\n", "------------", "----------", "-------------", "----");
      for (int i = 0; i < lim; i++)
        printf("%-12s␣␣%-10s␣␣%-13s␣␣%s\n", rows[i].time, rows[i].date, rows[i].standard, rows[i].meet);
      if (nrows ≡ 0) printf("(no␣times␣found)\n");
      putchar('\n');
    }
  }
```

28. Main program.

29. We define a small *Swimmer* record to hold the person-search parameters for each swimmer. Each entry supplies a *search_query* string sent to the database (ideally distinctive enough to return a small set), a *match_substr* (lower-cased) used to identify the correct row, and a *flag* bit used to filter output when the user passes a swimmer keyword to `-o`. Kalea's entry searches "Benavente" because the simple two-word query "Kalea Benavente" returns no results; the API requires an exact substring match against the registered name "Kalea Rose Benavente". Kenneth's entry uses "kenneth ray" and Keith's uses "keith santiago" to avoid false matches on the common surname "Evans".

```
⟨ Main function 29 ⟩ ≡
  typedef struct {
    const char *search_query;    /* Name string to search for */
    const char *match_substr;    /* Lower-case substring to match */
    int flag;    /* OPT_STELLA, OPT_KALEA, etc. */
  } Swimmer;
  static const Swimmer SWIMMERS[] = {{"Julianna␣Evans", "stella", OPT_STELLA},
    /* Stella Julianna Evans */
  {"Benavente", "kalea", OPT_KALEA},    /* Kalea Rose Benavente */
  {"Ray␣Evans", "kenneth␣ray", OPT_KENNY},    /* Kenneth Ray Evans */
  {"Santiago␣Evans", "keith␣santiago", OPT_KEITH}    /* Keith Santiago Evans */
  };
#define NUM_SWIMMERS ((int)(sizeof SWIMMERS/sizeof SWIMMERS[0]))
```

See also sections 30, 31, 32, and 33.

This code is used in section 7.

30. *parse_opts_str* tokenises a comma-separated option string (the argument to `-o`) and sets bits in *g_opts*. Unknown tokens are silently ignored so that future options can be added without breaking existing invocations.

```
⟨ Main function 29 ⟩ +≡
  static void parse_opts_str(const char *s)
  {    /* Work on a writable copy so strtok can insert NUL bytes. */
    char *buf = strdup(s);
    if (¬buf) return;
    char *tok = strtok(buf, ",");
    while (tok) {
      if (strcmp(tok, "stella") ≡ 0) g_opts |= OPT_STELLA;
      else if (strcmp(tok, "kalea") ≡ 0) g_opts |= OPT_KALEA;
      else if (strcmp(tok, "kenny") ≡ 0) g_opts |= OPT_KENNY;
      else if (strcmp(tok, "keith") ≡ 0) g_opts |= OPT_KEITH;
      else if (strcmp(tok, "fastest") ≡ 0) g_opts |= OPT_FASTEST;
      else if (strcmp(tok, "csv") ≡ 0) g_opts |= OPT_CSV;
      tok = strtok(Λ, ",");
    }
    free(buf);
  }
```

31. *parse_events_str* tokenises a comma-separated event-code string (the argument to `-e`) and appends each code to *g_events*. Codes that would overflow the `NUM_EVENTS`-entry array are silently dropped. The flag may be supplied multiple times; each invocation extends the list.

```
⟨ Main function 29 ⟩ +≡
  static void parse_events_str(const char *s)
  {
    char *buf = strdup(s);
    if (¬buf) return;
    char *tok = strtok(buf, ",");
    while (tok ∧ g_nevents < NUM_EVENTS) {
        /* Trim leading spaces left by comma-split with spaces around commas. */
      while (*tok ≡ '␣') tok++;
      strncpy(g_events[g_nevents], tok, sizeof g_events[0] − 1);
      g_events[g_nevents][sizeof g_events[0] − 1] = '\0';
      g_nevents++;
      tok = strtok(Λ, ",");
    }
    free(buf);
  }
```

32. *print_usage* writes the full usage message to standard error.

```
⟨ Main function 29 ⟩ +≡
  static void print_usage(const char *prog)
  {
    fprintf(stderr,
        "Usage:␣%s␣-o␣option[,option...]␣[-e␣event[,event...]]\n""\n""␣␣-o␣option,...␣␣␣c\
        omma-separated␣output␣options:\n""␣␣␣␣␣␣stella␣␣␣␣␣␣restrict␣output␣to␣S\
        tella␣Julianna␣Evans\n""␣␣␣␣␣␣␣kalea␣␣␣␣␣␣␣restrict␣output␣to␣K\
        alea␣Rose␣Benavente\n""␣␣␣␣␣␣␣kenny␣␣␣␣␣␣␣restrict␣output␣to␣K\
        enneth␣Ray␣Evans\n""␣␣␣␣␣␣␣keith␣␣␣␣␣␣␣restrict␣output␣to␣Keith␣Santiago␣Evans\\
        n""␣␣␣␣␣␣␣fastest␣␣␣␣␣print␣only␣the␣single␣fastest␣time␣per␣e\
        vent\n""␣␣␣␣␣␣␣csv␣␣␣␣␣␣␣␣␣emit␣CSV␣output␣(header␣+␣one␣line␣per␣t\
        ime)\n""\n""␣␣-e␣event,...␣␣␣␣restrict␣output␣to␣one␣or␣more␣SCY␣event␣\
        codes;\n""␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣comma-separated,␣or␣repeat␣-e␣for␣each␣ev\
        ent.\n""␣␣␣␣␣␣␣Valid␣codes:␣50␣FR␣SCY,␣100␣FR␣SCY,␣200␣FR␣SCY,␣500␣\
        FR␣SCY,\n""␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣50␣FL␣SCY,␣100␣FL␣SCY,␣50␣BK␣SCY,␣100␣B\
        K␣SCY,\n""␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣50␣BR␣SCY,␣100␣BR␣SCY,␣100␣IM␣SCY,␣200␣\
        IM␣SCY\n""\n""Examples:\n""␣␣%s␣-o␣stella,fastest\n""␣␣%s␣-o␣kalea,csv\n\
        ""␣␣%s␣-o␣kenny␣-e␣\"100␣FR␣SCY\"\n""␣␣%s␣-o␣keith␣-e␣\"100␣FR␣SCY,50␣FL␣SCY\
        \"\n""␣␣%s␣-o␣stella␣-e␣\"100␣FR␣SCY\"␣-e␣\"50␣FL␣SCY\"\n", prog, prog, prog, prog, prog,
        prog);
  }
```

33. We initialise the global `libcurl` state, parse the optional `-o` and `-e` flags, then for each swimmer (subject to swimmer-selection bits) resolve her *PersonKey* and iterate over events. If one or more `-e` codes were given only those events are fetched; otherwise all twelve are processed. In CSV mode a single header line is printed before the first data row. If no options at all are supplied, the usage message is printed and the program exits with status 1.

```
⟨ Main function 29 ⟩ +≡
  int main(int argc, char *argv[])
  {
    int ch;
    while ((ch = getopt(argc, argv, "o:e:")) ≠ −1) {
      switch (ch) {
      case 'o': parse_opts_str(optarg);
        break;
      case 'e': parse_events_str(optarg);
        break;
      default: print_usage(argv[0]);
        return 2;
      }
    }    /* Require at least one option; print usage and exit otherwise. */
    if (g_opts ≡ 0 ∧ g_nevents ≡ 0) {
      print_usage(argv[0]);
      return 1;
    }
    curl_global_init(CURL_GLOBAL_DEFAULT);    /* Print CSV header once, before any swimmer loop. */
    if (g_opts & OPT_CSV)
      printf("\"Swimmer\",\"Event\",\"Time\",\"Date\",\"Standard\",\"Meet\"\n");
    int swimmer_mask = OPT_STELLA | OPT_KALEA | OPT_KENNY | OPT_KEITH;
    for (int s = 0; s < NUM_SWIMMERS; s++) {
        /* Skip this swimmer if the -o flag restricts to the other one. */
      if ((g_opts & swimmer_mask) ∧ ¬(g_opts & SWIMMERS[s].flag)) continue;
      const char *name = Λ;
      char *key = lookup_person_key(SWIMMERS[s].search_query, SWIMMERS[s].match_substr, &name);
      if (¬key) {
        curl_global_cleanup();
        return 1;
      }
      if (¬(g_opts & OPT_CSV))
        printf("Swimmer:␣%s␣␣(PersonKey:␣%s)\n\n", name ? name : "(unknown)", key);
      if (g_nevents > 0) {    /* Fetch only the events requested via -e. */
        for (int i = 0; i < g_nevents; i++) fetch_times(key, g_events[i], name, g_opts);
      }
      else {
        for (int i = 0; i < NUM_EVENTS; i++) fetch_times(key, EVENTS[i], name, g_opts);
      }
      free(key);
      free((void *) name);
      if (¬(g_opts & OPT_CSV)) putchar('\n');
    }
    curl_global_cleanup();
    return 0;
  }
```

34. Glossary.

The following terms and interfaces appear throughout this program.

35.

JAQL (JSON Analytics Query Language). The query language used by the Sisense Analytic Engine to describe data requests. A JAQL query is a JSON object posted to the endpoint `/api/datasources/{name}/jaql`. Its `metadata` array specifies the columns (dimensions or measures) to retrieve; `filter` objects restrict row membership; and `count`/`offset` control pagination. The Sisense back-end translates JAQL to an internal columnar query, executes it against the ElastiCube, and returns results in a `values` array whose rows are parallel to the `metadata` array.

Sisense Analytic Engine (ElastiCube). A columnar, in-memory analytics database developed by Sisense. USA Swimming hosts a Sisense instance at `usaswimming.sisense.com` that powers the public data hub at `data.usaswimming.org`. The engine stores swim-time, meet, person, and time-standard data in compressed column stores called *ElastiCubes*; this program queries two of them—"Public Person Search" and "USA Swimming Times Elasticube"—via their JAQL endpoints, authenticating with a bearer token in the `Authorization` HTTP header.

36. Sisense JAQL API Calls. All requests are HTTP POST to `https://usaswimming.sisense.com/api/datasources/` with headers `Content-Type: application/json` and `Authorization: Bearer {token}`. The JSON request body has the following top-level fields:

- `datasource` (object). Identifies the ElastiCube to query.
  Fields:
  - `title` (string): human-readable name, e.g. `"Public Person Search"`.
  - `fullname` (string): internal path used by the server, e.g. `"LocalHost/Public Person Search"`.
- `metadata` (array of objects). Each element describes one column of the result.
  Per-element fields:
  - `jaql.table` (string): the ElastiCube table name (e.g. `"Persons"`, `"UsasSwimTime"`, `"Meet"`, `"SwimEvent"`, `"TimeStandard"`).
  - `jaql.column` (string): the column name within the table (e.g. `"FullName"`, `"PersonKey"`, `"SwimTimeFormatted"`, `"SortKey"`, `"MeetName"`, `"SeasonCalendarKey"`, `"EventCode"`, `"StandardType"`).
  - `jaql.dim` (string): the bracketed dimension path `"[Table.Column]"` used by the query engine.
  - `jaql.datatype` (string): `"text"` or `"numeric"`.
  - `jaql.title` (string): label for the column in the response.
  - `jaql.filter` (object, optional): restricts rows. This program uses two filter types: `{"contains": "..."}` for substring matches on text columns (person search) and `{"equals": value}` for exact matches (`PersonKey` and `EventCode` scope filters).
  - `panel` (string, optional): when set to `"scope"` the column acts as a *scope filter*—it narrows the result set without appearing as an output column.
- `count` (integer). Maximum number of rows to return. This program uses 10 for quick existence checks and 100 for full result sets.
- `offset` (integer). Zero-based row offset for pagination.

The response body is a JSON object. Relevant fields:

- `values` (array of arrays). Each inner array is one result row; its elements correspond positionally to the non-scope entries in `metadata`. Each element is an object with two keys: `data` (the raw value, number or string) and `text` (the formatted string representation). This program always reads the `text` field.
- `error` (boolean, present on failure). When true, `details` contains an error message.

Person search call. Endpoint: `.../aPublicIAAaPersonIAAaSearch/jaql`. Queries the `Persons` table with a `contains` filter on `FullName`. Returns `FullName` (text) and `PersonKey` (numeric) columns. This program sends `count:100` and scans rows until it finds one whose lower-cased name contains the match substring.

Times query call. Endpoint: `.../aUSAIAAaSwimmingIAAaTimesIAAaElasticube/jaql`. Uses two scope filters (`PersonKey` equals and `EventCode` equals) and retrieves five output columns in order: `SwimTimeFormatted` (formatted time string), `SortKey` (numeric sort key as decimal string, smaller = faster), `MeetName` (meet name), `SeasonCalendarKey` (swim date as `YYYYMMDD` integer), and `StandardType` from the `TimeStandard` table (motivational level: `"B"`, `"BB"`, `"A"`, `"Slower Than B"`, etc.).

37. libcurl API Calls. This program uses the libcurl "easy" interface for synchronous HTTP. All functions return a *CURLcode* (zero = `CURLE_OK`) except where noted.

- `curl_global_init(flags)`.

Parameter: `flags` (`long`) — a bitmask of subsystems to initialise. This program passes `CURL_GLOBAL_DEFAULT`, which enables SSL and the Windows socket layer on that platform. Must be called once before any other libcurl function. Returns a `CURLcode`; this program ignores the return value because failure is treated as fatal by the subsequent easy calls.

- `curl_global_cleanup(void)`. Releases all resources allocated by *curl_global_init*. Must be called once after all easy handles have been cleaned up. Returns nothing.
- `curl_easy_init(void)`. Allocates and returns a new easy handle (a `CURL *`). Returns `NULL` on failure. Each call to *post_json* creates its own handle and destroys it before returning, so handles are never shared between requests.
- `curl_easy_setopt(handle, option, value)`.

Parameters:

  - `handle` (`CURL *`): the easy handle.
  - `option` (`CURLoption`): a constant selecting the behaviour to configure. Options used here:
    `CURLOPT_URL` (`char *`) — the request URL.
    `CURLOPT_HTTPHEADER` (`struct curl_slist *`) — linked list of extra HTTP headers (`Content-Type` and `Authorization`).
    `CURLOPT_POSTFIELDS` (`char *`) — the POST body; setting this also switches the method to POST.
    `CURLOPT_WRITEFUNCTION` (function pointer) — callback invoked for each response chunk; signature `size_t cb(void*,size_t,size_t,void*)`.
    `CURLOPT_WRITEDATA` (`void *`) — the user-data pointer passed as the fourth argument to the write callback; here a pointer to the **Buffer** accumulator.
  - `value`: type depends on `option` (see above).
- `curl_easy_perform(handle)`.

Parameter: `handle` (`CURL *`). Executes the configured request synchronously, invoking the write callback for each received chunk. Returns `CURLE_OK` on success or a non-zero error code; on failure *post_json* frees the partial buffer and returns `NULL`.

- `curl_easy_cleanup(handle)`.

Parameter: `handle` (`CURL *`). Releases all resources associated with the handle. The handle must not be used after this call.

- `curl_slist_append(list, string)`.

Parameters:

  - `list` (`struct curl_slist *`): existing list head, or `NULL` to start a new list.
  - `string` (`const char *`): the string to append. Returns the new list head, or `NULL` on allocation failure. Used to build the two-header list (`Content-Type` then `Authorization`).
- `curl_slist_free_all(list)`.

Parameter: `list` (`struct curl_slist *`). Frees every node in the linked list. Called immediately after *curl_easy_perform* so the headers are released before the handle.

38. POSIX System Calls and Library Functions. The following identifiers from the POSIX.1-2008 standard are used directly in this program. Each entry gives the C signature, a description of each parameter, and a note on how the program uses it.

- `int fprintf(FILE *stream, const char *fmt, ...)`.

Parameters:
  - `stream`: destination file (`stderr` here).
  - `fmt`: printf-style format string.
  - `...`: values substituted into `fmt`. Writes formatted output to `stream`; returns the character count or a negative value on error. Used to report lookup and HTTP failures to standard error.
- `int fputs(const char *s, FILE *stream)`.

Parameters:
  - `s`: null-terminated string to write.
  - `stream`: destination file (`stderr` here). Writes `s` without a trailing newline; returns non-negative on success or `EOF` on error. Used for fixed error messages where no formatting is needed.
- `void free(void *ptr)`.

Parameter:
  - `ptr`: pointer to a heap block, or `NULL` (in which case nothing happens). Releases the block back to the heap. Called on every heap string (response buffers, duplicated names and keys) when they are no longer needed.
- `void *memcpy(void *dst, const void *src, size_t n)`.

Parameters:
  - `dst`: destination address.
  - `src`: source address.
  - `n`: number of bytes to copy. Copies exactly `n` bytes from `src` to `dst`; regions must not overlap. Returns `dst`. Used in *write_cb* to append each network chunk to the buffer, and in *scan_string* to copy a JSON string value.
- `int printf(const char *fmt, ...)`.

Parameters:
  - `fmt`: printf-style format string.
  - `...`: values substituted into `fmt`. Writes formatted output to standard output; returns the character count or a negative value on error. Used for all swimmer and time-table output.
- `int putchar(int c)`.

Parameter:
  - `c`: character value (as `unsigned char` cast to `int`). Writes one character to standard output; returns the character written, or `EOF` on error. Used to emit a blank line (`'\n'`) after each event section.
- `void *realloc(void *ptr, size_t size)`.

Parameters:
  - `ptr`: existing heap block, or `NULL`.
  - `size`: new size in bytes. Returns a pointer to the resized block (possibly moved), or `NULL` if allocation fails (the original block is unchanged on failure). When `ptr` is `NULL` the call is equivalent to `malloc`. Used in *write_cb* to grow the response buffer incrementally as libcurl delivers each chunk.
- `int snprintf(char *buf, size_t n, const char *fmt, ...)`.

Parameters:
  - `buf`: destination character array.
  - `n`: maximum bytes to write, including the null terminator.
  - `fmt`: printf-style format string.
  - `...`: values substituted into `fmt`. Writes at most `n`−1 formatted characters to `buf` and always null-terminates. Returns the number of characters that would have been written had the buffer been unlimited (so a return value ≥`n` signals truncation). Used to assemble URL strings, JSON bodies, and the bearer-token header.
- `char *strchr(const char *s, int c)`.

Parameters:

- `s`: string to search.
- `c`: character to find (compared as `unsigned char`). Returns a pointer to the first occurrence of `c` in `s`, including the terminator if `c` is `'\0'`, or `NULL`. Used in *scan_string* to find the closing double-quote of a JSON string value.

• `char *strdup(const char *s)`.
Parameter:
- `s`: null-terminated string to duplicate. Allocates a new heap block of `strlen(s)+1` bytes, copies `s` into it, and returns the pointer; returns `NULL` on failure. Used in *lookup_person_key* to persist the swimmer's full name and PersonKey string across the lifetime of a query.

• `size_t strlen(const char *s)`.
Parameter:
- `s`: null-terminated string. Returns the number of bytes before the null terminator. Used to compute loop bounds when lower-casing names, to advance past a search needle in *scan_string* and *scan_long*, and to validate the 8-character date string in *format_date*.

• `char *strncpy(char *dst, const char *src, size_t n)`.
Parameters:
- `dst`: destination array (at least `n` bytes).
- `src`: source string.
- `n`: maximum bytes to copy. Copies up to `n` bytes; if `src` is shorter than `n`, the remainder of `dst` is zero-filled. If `src` is at least `n` bytes long, `dst` will *not* be null-terminated. Returns `dst`. This program always writes `dst[sizeof dst - 1] = '\0'` after the call to guarantee termination. Used to copy time, standard, and meet strings into `TimeRow`.

• `char *strstr(const char *hay, const char *needle)`.
Parameters:
- `hay`: string to search within.
- `needle`: substring to search for. Returns a pointer to the first occurrence of `needle` in `hay`, or `NULL` if not found. The workhorse of the JSON scanner: used to locate key names (`"text"`, `"data"`, `"values"`) and to advance through the raw Sisense response text.

• `double strtod(const char *s, char **endptr)`.
Parameters:
- `s`: string containing a floating-point number.
- `endptr`: if non-`NULL`, receives a pointer to the first character not consumed by the conversion. Returns the parsed `double`; sets `*endptr` past the converted text. Used to convert the Sisense sort-key string (e.g. `"1010007029.00"`) to a `double` for the insertion sort.

• `long strtol(const char *s, char **endptr, int base)`.
Parameters:
- `s`: string containing an integer.
- `endptr`: if non-`NULL`, receives a pointer to the first character not consumed.
- `base`: numeric base (2–36), or 0 for auto-detection from a `0x` or `0` prefix. This program passes 10 (decimal). Returns the parsed `long`; sets `*endptr` past the converted text. Used in *scan_long* to read the `PersonKey` integer from the JSON person-search response.

39. Index.

argc: 33.
argv: 33.
auth_hdr: 17.
b: 16.
body: 17, 23, 27.
buf: 17, 30, 31.
Buffer: 13, 16, 17, 37.
ch: 33.
curl: 17.
CURL: 17.
curl_easy_cleanup: 17.
curl_easy_init: 17.
curl_easy_perform: 17, 37.
curl_easy_setopt: 17.
curl_global_cleanup: 33.
CURL_GLOBAL_DEFAULT: 33.
curl_global_init: 33, 37.
curl_slist: 17.
curl_slist_append: 17.
curl_slist_free_all: 17.
CURLcode: 17, 37.
CURLE_OK: 17.
CURLOPT_HTTPHEADER: 17.
CURLOPT_POSTFIELDS: 17.
CURLOPT_URL: 17.
CURLOPT_WRITEDATA: 17.
CURLOPT_WRITEFUNCTION: 17.
data: 13, 16, 17.
date: 14, 27.
date_str: 27.
e: 20.
end: 21.
event_code: 27.
EVENTS: 10, 33.
fetch_times: 27, 33.
flag: 29, 33.
flen: 23.
format_date: 26, 27, 38.
fprintf: 23, 27, 32.
fputs: 23.
free: 17, 23, 27, 30, 31, 33.
full_name: 23.
g_events: 11, 31, 33.
g_nevents: 11, 31, 33.
g_opts: 11, 30, 33.
getopt: 33.
hdrs: 17.
i: 23, 25, 27, 33.
insertion_sort: 25, 27.
j: 25.
key: 20, 21, 23, 33.
len: 20.
lim: 27.
lookup_person_key: 22, 23, 33, 38.
lower: 23.
main: 33.
match_substr: 22, 23, 29, 33.
max_len: 20.
MAX_TIMES: 10, 27.
meet: 14, 27.
meet_str: 27.
memcpy: 16, 20.
n: 16, 25.
name: 23, 33.
needle: 20, 21.
nmemb: 16.
nrows: 27.
NUM_EVENTS: 10, 11, 31, 33.
NUM_SWIMMERS: 29, 33.
OPT_CSV: 11, 27, 30, 33.
OPT_FASTEST: 11, 27, 30.
OPT_KALEA: 11, 29, 30, 33.
OPT_KEITH: 11, 29, 30, 33.
OPT_KENNY: 11, 29, 30, 33.
OPT_STELLA: 11, 29, 30, 33.
optarg: 33.
opts: 27.
out: 20, 21, 26.
out_name: 22, 23.
p: 16, 20, 21, 23, 27.
parse_events_str: 31, 33.
parse_opts_str: 30, 33.
person_key: 27.
PersonKey: 2, 22, 33.
pk: 23.
pos: 20, 21.
post_json: 17, 23, 27, 37.
print_usage: 32, 33.
printf: 27, 33.
prog: 32.
ptr: 16.
putchar: 27, 33.
rc: 17.
realloc: 16.
resp: 23, 27.
rows: 25, 27.
s: 30, 31, 33.
scan_long: 21, 23, 38.
scan_string: 20, 23, 27, 38.
search_query: 22, 23, 29, 33.
SISENSE_API: 10, 23, 27.
SISENSE_TOKEN: 10, 17.

size: 13, 16.
snprintf: 17, 20, 21, 23, 27.
sort_key: 14, 25, 27.
sort_str: 27.
standard: 14, 27.
std_str: 27.
stderr: 23, 27, 32.
strchr: 20.
strcmp: 30.
strdup: 23, 30, 31.
strlen: 20, 21, 23, 26.
strncpy: 26, 27, 31.
strstr: 20, 21, 23, 27.
strtod: 27.
strtok: 30, 31.
strtol: 21.
Swimmer: 29.
swimmer_mask: 33.
swimmer_name: 27.
`SWIMMERS`: 29, 33.
time: 14, 27.
time_str: 27.
TimeRow: 14, 25, 27.
tmp: 23, 25.
tok: 30, 31.
ud: 16.
url: 17, 23, 27.
write_cb: 16, 17, 38.
ymd8: 26.

⟨ Global constants 10 ⟩ Used in section 7.
⟨ HTTP utilities 16, 17 ⟩ Used in section 7.
⟨ Includes 9 ⟩ Used in section 7.
⟨ JSON scanner 20, 21 ⟩ Used in section 7.
⟨ Main function 29, 30, 31, 32, 33 ⟩ Used in section 7.
⟨ Option flags 11 ⟩ Used in section 7.
⟨ Person lookup 23 ⟩ Used in section 7.
⟨ Times fetch 25, 26, 27 ⟩ Used in section 7.
⟨ Type definitions 13, 14 ⟩ Used in section 7.

STELLA-TIMES

	Section	Page
Swim Times	1	1
Includes and constants	8	3
Data structures	12	4
HTTP utilities	15	5
JSON scanner	18	6
Person lookup	22	8
Times fetch	24	11
Main program	28	14
Glossary	34	17
Index	39	22